

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	A Syed Thoufiq	Reg. No.:	192424211
Section / Batch:	Slot-D/2024	Date:	9/8/26

Code of Conduct

I A Syed Thoufiq(192424211) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A Syed Thoufiq

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging, HMM	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q1

Annexure A – SCENARIO BASED

A company is developing an NLP-based grammar checker for educational applications. The system is trained using a manually POS-tagged corpus. Your task is to recreate the Hidden Markov Model (HMM) **without using any built-in NLP or HMM libraries**.

From the given corpus,

- determine the **Emission Probability**
- determine the **Transition Probability**
- implement the **Viterbi Algorithm**
- predict the POS tags for the new input sentence.

Use only Python dictionaries, loops, and basic programming constructs.

Training Corpus

Sentence	POS Tags
The/DT boy/NN eats/VBZ rice/NN	DT NN VBZ NN
The/DT girl/NN drinks/VBZ milk/NN	DT NN VBZ NN
A/DT cat/NN drinks/VBZ milk/NN	DT NN VBZ NN
The/DT dog/NN chases/VBZ cat/NN	DT NN VBZ NN
A/DT teacher/NN teaches/VBZ students/NNS	DT NN VBZ NNS
Students/NNS study/VBP English/NN	NNS VBP NN
Birds/NNS fly/VBP high/RB	NNS VBP RB
Children/NNS play/VBP games/NNS	NNS VBP NNS

New Input Sentence

The cat drinks milk

Tasks

1. Separate each sentence into words and POS tags.
2. Compute the **Emission Probability** of every word.
3. Compute the **Transition Probability** between POS tags.
4. Recreate the Hidden Markov Model using Python dictionaries.
5. Implement the **Viterbi Algorithm** without using any built-in HMM/NLP libraries.
6. Predict the POS tag sequence for “The cat drinks milk”

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1	20	15	25	15	12	87	87

Faculty In-charge	Course Coordinator	Program Director
Dr. T. Kumaragurubaran		
Signature: Kumaragurubaran T	Signature: _____	Signature: _____
Date: 9/8/26	Date: _____	Date: _____

Annexure A – Scenario Based: HMM POS Tagging

Objective:

Recreate a Hidden Markov Model (HMM) for POS tagging without using built-in NLP or HMM libraries, calculate emission and transition probabilities, implement the Viterbi algorithm, and predict POS tags for “The cat drinks milk”.

1. Separation of Words and POS Tags

Sentence	Words	POS Tags
1	The, boy, eats, rice	DT, NN, VBZ, NN
2	The, girl, drinks, milk	DT, NN, VBZ, NN
3	A, cat, drinks, milk	DT, NN, VBZ, NN
4	The, dog, chases, cat	DT, NN, VBZ, NN
5	A, teacher, teaches, students	DT, NN, VBZ, NNS
6	Students, study, English	NNS, VBP, NN
7	Birds, fly, high	NNS, VBP, RB
8	Children, play, games	NNS, VBP, NNS

POS states: DT, NN, VBZ, NNS, VBP, RB.

2. Emission Probability

Formula: $P(\text{word}|\text{tag}) = \text{Count}(\text{word}, \text{tag}) / \text{Count}(\text{tag})$

POS	Word	Count	P(Word POS)
DT	The	3	0.60
DT	A	2	0.40
NN	boy	1	0.10
NN	girl	1	0.10
NN	rice	1	0.10
NN	milk	2	0.20
NN	cat	2	0.20
NN	dog	1	0.10
NN	teacher	1	0.10
NN	English	1	0.10
VBZ	eats	1	0.20
VBZ	drinks	2	0.40

VBZ	chases	1	0.20
VBZ	teaches	1	0.20
NNS	students	1	0.20
NNS	Students	1	0.20
NNS	Birds	1	0.20
NNS	Children	1	0.20
NNS	games	1	0.20
VBP	study	1	0.3333
VBP	fly	1	0.3333
VBP	play	1	0.3333
RB	high	1	1.00

3. Transition Probability

Formula: $P(\text{Tag2}|\text{Tag1}) = \text{Count}(\text{Tag1} \rightarrow \text{Tag2}) / \text{Count of transitions from Tag1}$

From	To	Probability
DT	NN	1.00
NN	VBZ	1.00
VBZ	NN	0.80
VBZ	NNS	0.20
NNS	VBP	1.00
VBP	NN	0.3333
VBP	RB	0.3333
VBP	NNS	0.3333

All other transitions have probability 0.

4. Initial Probability

There are 8 training sentences:

1. DT starts 5 sentences: $P(\text{DT}) = 5/8 = 0.625$
2. NNS starts 3 sentences: $P(\text{NNS}) = 3/8 = 0.375$
3. All other initial probabilities are 0.

5. HMM Implementation Using Python Dictionaries

Training corpus

```
corpus = [
    [("The", "DT"), ("boy", "NN"), ("eats", "VBZ"), ("rice", "NN")],
    [("The", "DT"), ("girl", "NN"), ("drinks", "VBZ"), ("milk", "NN")],
```

```
[("A", "DT"), ("cat", "NN"), ("drinks", "VBZ"), ("milk", "NN")],
[("The", "DT"), ("dog", "NN"), ("chases", "VBZ"), ("cat", "NN")],
[("A", "DT"), ("teacher", "NN"), ("teaches", "VBZ"), ("students", "NNS")],
[("Students", "NNS"), ("study", "VBP"), ("English", "NN")],
[("Birds", "NNS"), ("fly", "VBP"), ("high", "RB")],
[("Children", "NNS"), ("play", "VBP"), ("games", "NNS")]
]
```

```
tag_count = {}
```

```
emission_count = {}
```

```
for sentence in corpus:
```

```
    for word, tag in sentence:
```

```
        if tag not in tag_count:
```

```
            tag_count[tag] = 0
```

```
        tag_count[tag] += 1
```

```
    key = (word, tag)
```

```
    if key not in emission_count:
```

```
        emission_count[key] = 0
```

```
    emission_count[key] += 1
```

```
emission = {}
```

```
for key in emission_count:
```

```
    tag = key[1]
```

```
    emission[key] = emission_count[key] / tag_count[tag]
```

```
transition_count = {}
```

```
from_count = {}
```

```
for sentence in corpus:
```

```
    for i in range(len(sentence) - 1):
```

```
        tag1 = sentence[i][1]
```

```
        tag2 = sentence[i + 1][1]
```

```
        key = (tag1, tag2)
```

```
if key not in transition_count:
```

```
    transition_count[key] = 0
```

```
transition_count[key] += 1
```

```
if tag1 not in from_count:
```

```
    from_count[tag1] = 0
```

```
from_count[tag1] += 1
```

```
transition = {}
```

```
for key in transition_count:
```

```
    transition[key] = transition_count[key] / from_count[key[0]]
```

```
initial_count = {}
```

```
for sentence in corpus:
```

```
    first_tag = sentence[0][1]
```

```
    if first_tag not in initial_count:
```

```
        initial_count[first_tag] = 0
```

```
    initial_count[first_tag] += 1
```

```
initial = {}
```

```
for tag in initial_count:
```

```
    initial[tag] = initial_count[tag] / len(corpus)
```

6. Viterbi Algorithm

The Viterbi algorithm finds the most probable sequence of hidden POS tags. For each word, it considers the previous POS tag, transition probability, and emission probability.

$$V_t(\text{tag}) = \max[V_{(t-1)}(\text{previous tag}) \times P(\text{tag}|\text{previous tag}) \times P(\text{word}|\text{tag})]$$

7. Viterbi Calculation for the New Sentence

Word	Selected POS	Calculation	Probability
The	DT	0.625×0.60	0.375
cat	NN	$0.375 \times 1.00 \times 0.20$	0.075
drinks	VBZ	$0.075 \times 1.00 \times 0.40$	0.030
milk	NN	$0.030 \times 0.80 \times 0.20$	0.0048

8. Final Prediction

Word	Predicted POS
------	---------------

The	DT
cat	NN
drinks	VBZ
milk	NN

Final POS sequence: DT NN VBZ NN

Conclusion:

The manually constructed HMM uses emission probabilities to determine how likely a word is for a POS tag and transition probabilities to determine how likely one POS tag is to follow another. The Viterbi algorithm selects the most probable sequence, DT → NN → VBZ → NN, for “The cat drinks milk”.

CODE:

HMM POS TAGGING USING VITERBI ALGORITHM

```
corpus = [
    [("The", "DT"), ("boy", "NN"), ("eats", "VBZ"), ("rice", "NN")],
    [("The", "DT"), ("girl", "NN"), ("drinks", "VBZ"), ("milk", "NN")],
    [("A", "DT"), ("cat", "NN"), ("drinks", "VBZ"), ("milk", "NN")],
    [("The", "DT"), ("dog", "NN"), ("chases", "VBZ"), ("cat", "NN")],
    [("A", "DT"), ("teacher", "NN"), ("teaches", "VBZ"), ("students", "NNS")],
    [("Students", "NNS"), ("study", "VBP"), ("English", "NN")],
    [("Birds", "NNS"), ("fly", "VBP"), ("high", "RB")],
    [("Children", "NNS"), ("play", "VBP"), ("games", "NNS")]
]
# Separate words and tags
print("WORDS AND POS TAGS")
for i, sentence in enumerate(corpus, 1):
    words = []
    tags = []
    for word, tag in sentence:
        words.append(word)
        tags.append(tag)
    print("\nSentence", i)
    print("Words:", words)
    print("Tags :", tags)
# Get POS states
states = []
for sentence in corpus:
    for word, tag in sentence:
        if tag not in states:
            states.append(tag)
print("\nPOS TAGS:", states)
# Count POS tags
tag_count = {}
for sentence in corpus:
    for word, tag in sentence:
```



```

    if tag not in tag_count:
        tag_count[tag] = 0
    tag_count[tag] += 1
print("\nPOS TAG COUNTS")
for tag in tag_count:
    print(tag, "=", tag_count[tag])
# Emission probabilities

emission_count = {}
for sentence in corpus:
    for word, tag in sentence:
        key = (word, tag)
        if key not in emission_count:
            emission_count[key] = 0
        emission_count[key] += 1
emission = {}
for key in emission_count:
    tag = key[1]
    emission[key] = emission_count[key] / tag_count[tag]
print("\nEMISSION PROBABILITIES")
for key in emission:
    print(
        key,
        "=",
        round(emission[key], 4)
    )
# Transition probabilities
transition_count = {}
from_tag_count = {}
for sentence in corpus:
    for i in range(len(sentence) - 1):
        current_tag = sentence[i][1]
        next_tag = sentence[i + 1][1]
        key = (current_tag, next_tag)
        if key not in transition_count:
            transition_count[key] = 0
        transition_count[key] += 1
        if current_tag not in from_tag_count:
            from_tag_count[current_tag] = 0
        from_tag_count[current_tag] += 1
transition = {}
for key in transition_count:
    current_tag = key[0]
    transition[key] = (
        transition_count[key] /
        from_tag_count[current_tag]
    )
print("\nTRANSITION PROBABILITIES")
for key in transition:
    print(
        key,
        "=",
        round(transition[key], 4)
    )

```

```

    )
# Initial probabilities
initial_count = {}
for sentence in corpus:
    first_tag = sentence[0][1]
    if first_tag not in initial_count:
        initial_count[first_tag] = 0
    initial_count[first_tag] += 1
initial = {}
for tag in initial_count:
    initial[tag] = initial_count[tag] / len(corpus)
print("\nINITIAL PROBABILITIES")
for tag in initial:
    print(
        tag,
        "=",
        round(initial[tag], 4)
    )
# Viterbi Algorithm
sentence = ["The", "cat", "drinks", "milk"]
viterbi = []
backpointer = []
first_word = sentence[0]

v = {}
b = {}
for state in states:
    start_probability = initial.get(state, 0)
    emission_probability = emission.get(
        (first_word, state), 0
    )
    v[state] = (
        start_probability *
        emission_probability
    )
    b[state] = None
viterbi.append(v)
backpointer.append(b)
for i in range(1, len(sentence)):
    word = sentence[i]
    v = {}
    b = {}
    for current_state in states:
        best_probability = 0
        best_previous_state = None
        emission_probability = emission.get(
            (word, current_state), 0
        )
        for previous_state in states:
            transition_probability = transition.get(
                (previous_state, current_state), 0
            )
            probability = (

```

```

        viterbi[i - 1][previous_state]
        * transition_probability
        * emission_probability
    )
    if probability > best_probability:
        best_probability = probability
        best_previous_state = previous_state
    v[current_state] = best_probability
    b[current_state] = best_previous_state
    viterbi.append(v)
    backpointer.append(b)
# Find best final state
best_state = None
best_probability = 0
for state in states:
    if viterbi[-1][state] > best_probability:
        best_probability = viterbi[-1][state]
        best_state = state
# Backtracking
best_path = [best_state]
for i in range(len(sentence) - 1, 0, -1):
    previous_state = backpointer[i][best_path[0]]
    best_path.insert(0, previous_state)
# Final output
print("\nVITERBI RESULT")
for i in range(len(sentence)):
    print(sentence[i], "->", best_path[i])
print("\nFinal POS Sequence:")
print(best_path)
print("\nFinal Viterbi Probability:")
print(best_probability)
print("\nTagged Sentence:")
for i in range(len(sentence)):
    print(
        sentence[i] + "/" + best_path[i],
        end=" "
    )
)

```

Output:

WORDS AND POS TAGS

Sentence 1

Words: ['The', 'boy', 'eats', 'rice']

Tags : ['DT', 'NN', 'VBZ', 'NN']

Sentence 2

Words: ['The', 'girl', 'drinks', 'milk']

Tags : ['DT', 'NN', 'VBZ', 'NN']

Sentence 3

Words: ['A', 'cat', 'drinks', 'milk']

Tags : ['DT', 'NN', 'VBZ', 'NN']

Sentence 4

Words: ['The', 'dog', 'chases', 'cat']

Tags : ['DT', 'NN', 'VBZ', 'NN']

Sentence 5

Words: ['A', 'teacher', 'teaches', 'students']

Tags : ['DT', 'NN', 'VBZ', 'NNS']

Sentence 6

Words: ['Students', 'study', 'English']

Tags : ['NNS', 'VBP', 'NN']

Sentence 7

Words: ['Birds', 'fly', 'high']

Tags : ['NNS', 'VBP', 'RB']

Sentence 8

Words: ['Children', 'play', 'games']

Tags : ['NNS', 'VBP', 'NNS']

POS TAGS: ['DT', 'NN', 'VBZ', 'NNS', 'VBP', 'RB']

POS TAG COUNTS

DT = 5

NN = 10

VBZ = 5

NNS = 5

VBP = 3

RB = 1

EMISSION PROBABILITIES

('The', 'DT') = 0.6

('boy', 'NN') = 0.1

('eats', 'VBZ') = 0.2

('rice', 'NN') = 0.1

('girl', 'NN') = 0.1

('drinks', 'VBZ') = 0.4

('milk', 'NN') = 0.2

('A', 'DT') = 0.4

('cat', 'NN') = 0.2

('dog', 'NN') = 0.1

('chases', 'VBZ') = 0.2

('teacher', 'NN') = 0.1

('teaches', 'VBZ') = 0.2

('students', 'NNS') = 0.2

('Students', 'NNS') = 0.2

('study', 'VBP') = 0.3333

('English', 'NN') = 0.1

('Birds', 'NNS') = 0.2

('fly', 'VBP') = 0.3333

('high', 'RB') = 1.0

('Children', 'NNS') = 0.2

('play', 'VBP') = 0.3333

('games', 'NNS') = 0.2

TRANSITION PROBABILITIES

('DT', 'NN') = 1.0

('NN', 'VBZ') = 1.0

('VBZ', 'NN') = 0.8

('VBZ', 'NNS') = 0.2

('NNS', 'VBP') = 1.0

('VBP', 'NN') = 0.3333

('VBP', 'RB') = 0.3333

('VBP', 'NNS') = 0.3333

INITIAL PROBABILITIES

DT = 0.625

NNS = 0.375

VITERBI RESULT

The -> DT

cat -> NN

drinks -> VBZ

milk -> NN

Final POS Sequence:

['DT', 'NN', 'VBZ', 'NN']

Final Viterbi Probability:

0.0048000000000000002

Tagged Sentence:

The/DT cat/NN drinks/VBZ milk/NN

```
PS C:\Users\lenovo\OneDrive\Documents\DSA0308> c::; cd 'c:\Users\lenovo\OneDrive\Documents\DSA0308'; & 'C:\Users\lenovo\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\lenovo\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '50340' '--' 'c:\Users\lenovo\OneDrive\Documents\DSA0308\NLP _ C02-AT2_H#M POS TAGGING USING VITERBI ALGORITHM.py'
```

Sentence 1

Words: ['The', 'boy', 'eats', 'rice']

Tags : ['DT', 'NN', 'VBZ', 'NN']

Sentence 2

Words: ['The', 'girl', 'drinks', 'milk']

Tags : ['DT', 'NN', 'VBZ', 'NN']

Sentence 3

Words: ['A', 'cat', 'drinks', 'milk']

Tags : ['DT', 'NN', 'VBZ', 'NN']

Sentence 4

Words: ['The', 'dog', 'chases', 'cat']

Tags : ['DT', 'NN', 'VBZ', 'NN']

Sentence 5

Words: ['A', 'teacher', 'teaches', 'students']

Tags : ['DT', 'NN', 'VBZ', 'NNS']

Sentence 6

Words: ['Students', 'study', 'English']

Tags : ['NNS', 'VBP', 'NN']

Sentence 7

Words: ['Birds', 'fly', 'high']

Tags : ['NNS', 'VBP', 'RB']

Sentence 8

Words: ['Children', 'play', 'games']

Tags : ['NNS', 'VBP', 'NNS']

POS TAGS: ['DT', 'NN', 'VBZ', 'NNS', 'VBP', 'RB']

```
PS C:\Users\lenovo\OneDrive\Documents\DSA0308> c:: cd 'c:\Users\lenovo\OneDrive\Documents\DSA0308'; & 'C:\Users\lenovo\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\lenovo\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '50340' '--' 'c:\Users\lenovo\OneDrive\Documents\DSA0308\NLP _ CO2-AT2_HMM POS TAGGING USING VITERBI ALGORITHM.py'
```

POS TAG COUNTS

```
DT = 5
NN = 10
VBZ = 5
NNS = 5
VBP = 3
RB = 1
```

EMISSION PROBABILITIES

```
('The', 'DT') = 0.6
('boy', 'NN') = 0.1
('eats', 'VBZ') = 0.2
('rice', 'NN') = 0.1
('girl', 'NN') = 0.1
('drinks', 'VBZ') = 0.4
('milk', 'NN') = 0.2
('A', 'DT') = 0.4
('cat', 'NN') = 0.2
('dog', 'NN') = 0.1
('chases', 'VBZ') = 0.2
('teacher', 'NN') = 0.1
('teaches', 'VBZ') = 0.2
('students', 'NNS') = 0.2
('Students', 'NNS') = 0.2
('study', 'VBP') = 0.3333
('English', 'NN') = 0.1
('Birds', 'NNS') = 0.2
('fly', 'VBP') = 0.3333
('high', 'RB') = 1.0
('Children', 'NNS') = 0.2
('play', 'VBP') = 0.3333
('games', 'NNS') = 0.2
```

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS SONARQUE 4

```
PS C:\Users\lenovo\OneDrive\Documents\DSA0308> c:: cd 'c:\Users\lenovo\OneDrive\Documents\DSA0308'; & 'C:\Users\lenovo\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\lenovo\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '50340' '--' 'c:\Users\lenovo\OneDrive\Documents\DSA0308\NLP _ CO2-AT2_HMM POS TAGGING USING VITERBI ALGORITHM.py'
```

TRANSITION PROBABILITIES

```
('DT', 'NN') = 1.0
('NN', 'VBZ') = 1.0
('VBZ', 'NN') = 0.8
('VBZ', 'NNS') = 0.2
('NNS', 'VBP') = 1.0
('VBP', 'NN') = 0.3333
('VBP', 'RB') = 0.3333
('VBP', 'NNS') = 0.3333
```

INITIAL PROBABILITIES

```
DT = 0.625
NNS = 0.375
```

VITERBI RESULT

```
The -> DT
cat -> NN
drinks -> VBZ
milk -> NN
```

Final POS Sequence:

```
['DT', 'NN', 'VBZ', 'NN']
```

Final Viterbi Probability:

```
0.004800000000000002
```

Tagged Sentence:

```
The/DT cat/NN drinks/VBZ milk/NN
```

```
PS C:\Users\lenovo\OneDrive\Documents\DSA0308>
```